LAPORAN PRAKTIKUM PEKAN 1

PEMOGRAMAN BERORIENTASI OBJEK

Class, Object,Constructor, dan Method

Interaksi Objek & Manipulasi State (Mini Bank)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah,S.ST., M.Kom.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 1 mata kuliah Pemrograman Berorientasi Objek dengan materi Class, Object, Constructor, dan Method serta Interaksi Objek dan Manipulasi State (Mini Bank) dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Melalui praktikum ini, penulis mempelajari konsep dasar pemrograman berorientasi objek, khususnya mengenai class, object, constructor, dan method, serta penerapannya dalam membangun program sederhana berupa Mini Bank. Praktikum ini juga membantu penulis memahami bagaimana objek dapat saling berinteraksi dan bagaimana state suatu objek dapat dimanipulasi melalui method.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dalam penyusunan maupun penyajian materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan evaluasi dan perbaikan untuk laporan maupun praktikum selanjutnya.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai konsep dasar Pemrograman Berorientasi Objek, khususnya dalam penerapan class, object, constructor, method, serta interaksi dan manipulasi state pada sebuah program.

Padang, 8 September 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman merupakan salah satu bagian penting dalam ilmu komputer yang digunakan untuk mengembangkan berbagai aplikasi dan sistem. Salah satu pendekatan pemrograman yang banyak digunakan adalah Pemrograman Berorientasi Objek (PBO). Pemrograman Berorientasi Objek merupakan paradigma pemrograman yang menggunakan objek sebagai dasar dalam membangun sebuah program. Dengan pendekatan ini, program dapat dibuat lebih terstruktur, mudah dikembangkan, dan lebih mudah dipahami.

Dalam Pemrograman Berorientasi Objek, terdapat beberapa konsep dasar yang perlu dipahami, seperti class, object, constructor, dan method. Class digunakan sebagai rancangan atau cetak biru untuk membentuk suatu objek, sedangkan object merupakan bentuk nyata dari class yang memiliki atribut dan perilaku tertentu. Constructor digunakan untuk melakukan inisialisasi objek ketika objek dibuat, sementara method digunakan untuk mendefinisikan tindakan atau perilaku yang dapat dilakukan oleh suatu objek.

Selain memahami konsep dasar tersebut, interaksi antarobjek dan manipulasi state juga merupakan bagian penting dalam Pemrograman Berorientasi Objek. State merupakan kondisi atau nilai yang dimiliki oleh suatu objek dan dapat berubah melalui method tertentu. Penerapan konsep tersebut dapat digambarkan melalui pembuatan program sederhana seperti Mini Bank, di mana objek dapat memiliki data rekening, melakukan transaksi, serta mengubah keadaan atau nilai yang dimilikinya.

Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep dasar Pemrograman Berorientasi Objek, khususnya class, object, constructor, dan method, serta mampu menerapkan konsep interaksi objek dan manipulasi state dalam bahasa pemrograman Java melalui pembuatan program Mini Bank.

1.2 Tujuan

1. Mendefinisikan class sebagai model dari entitas dunia nyata
2. Mendefisikan object dan memahami alokasi memori independent
3. Menggunakan constructor untuk inisialisasi awal state
4. Memanipulasi state (atribut) melalui behavior (method) menggunakan interaksi CLI

1.3 Manfaat

1. Memahami konsep class sebagai model atau rancangan dari suatu entitas dunia nyata sehingga dapat digunakan sebagai dasar dalam pembuatan program berorientasi objek.
2. Memahami konsep object serta mengetahui bahwa setiap object memiliki alokasi memori yang independent, sehingga setiap object dapat memiliki state atau nilai atribut yang berbeda.
3. Memahami penggunaan constructor untuk memberikan nilai awal atau melakukan inisialisasi state pada saat object dibuat.
4. Memahami manipulasi state melalui method serta mengetahui bagaimana suatu object dapat menjalankan behavior tertentu melalui interaksi berbasis Command Line Interface (CLI).
5. Meningkatkan kemampuan dalam menerapkan konsep Pemrograman Berorientasi Objek melalui pembuatan program sederhana yang menggambarkan interaksi antarobject dan perubahan state.

BAB II

PEMBAHASAN

2.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object.

• **Class sebagai Blueprint**, Menurut Deitel & Deitel (2017) dalam bukunya Java: How to Program, sebuah class adalah cetak biru (blueprint) dari sebuah objek. Class mendefinisikan struktur data (attributes/state) dan sekumpulan operasi (methods/behavior) yang dapat dilakukan pada data tersebut. Class sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi.

• **Object dan State**, Objek adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer. Setiap objek memiliki state/kondisi yang independen. Sierra & Bates (2005) dalam Head First Java mengilustrasikan bahwa dua objek dari class yang sama misalnya Rekening dapat memiliki nilai state seperti saldo yang sama sekali berbeda, dan perubahan pada satu objek tidak akan memengaruhi objek lainnya.

• **Constructor** adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya adalah untuk menginisialisasi state awal dari objek tersebut (Oracle, 2023).

Dalam konteks perbankan, Class adalah konsep "Rekening", Object adalah "Rekening milik Mawar" atau "Rekening milik Melati", State adalah "Saldo saat ini", dan Behavior adalah aktivitas seperti "Setor Tunai" yang akan memanipulasi state saldo tersebut.

2.2 Langkah Langkah

1. **Membuat Class Rekening**
    1) Pertama saya membuat Package “MiniBank”
    2) Selanjutnya dilanjukan dengan membuat Class Rekening, Berikut Screenshot lengkap nya

```java
package MiniBank;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik + " Berhasil dibuat dengan saldo Rp" + saldo);
}

public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " Berhasil Saldo saat ini: Rp" + saldo);
    }else {
        System.out.println("Gagal: Nominal setor Harus lebih dari 0! ");
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No.Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("---------------");

}
}
```

Penjelasan :

1) Buat class Rekening

```java
public class Rekening {
```

Ini mengartikan kita sedang membuat sebuah class bernama Rekening. Class bisa dianggap sebagai cetakan/model untuk membuat objek rekening.

2) Menambahkan State/Atribut

```java
String nomorRekening;
String namaPemilik;
double saldo;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class rekening ini tersimpan data berupa

- nomor rekening dengan tipe data string
- nama pemilik dengan tipe data string
- saldo dengan tipe data double.

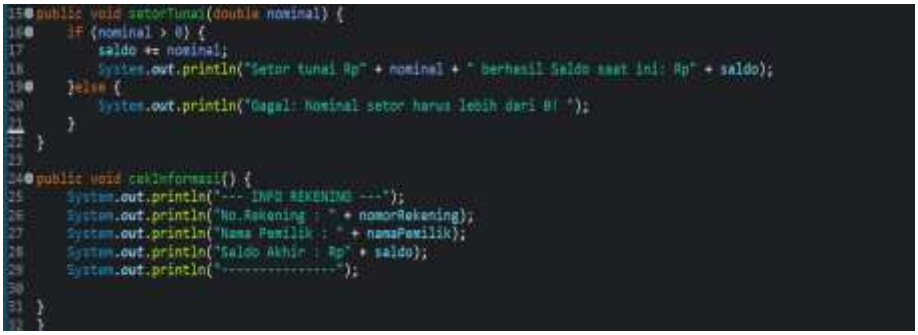
3) Buat Constructor

```java
public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik + " Berhasil dibuat dengan saldo Rp" + saldo);
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek.

- nomorRekening di inisialisasikan dengan nomor
- namaPemilik dengan nama
- saldo dengan saldo awal.
- Program akan menghasilkan output “Rekening atas nama” (berisi nama dari pemilik) Berhasil dibuat dengan saldo Rp (berisa nominal saldo dari pemilik rekening).

4) Buat Method/Behavior



```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " berhasil Saldo saat ini: Rp" + saldo);
    }else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0! ");
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No.Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("---------------");

}
}
```

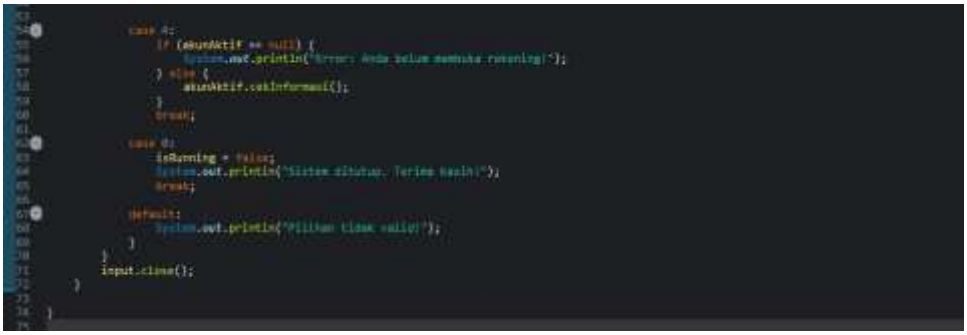
Dibagian sini Untuk Methodnya baru ada setor tunai dan cek informasi.

- Method Setor tunai itu menjelaskan “Jika nominal nya besar dari 0 maka saldo akan bertambah sebanyak nominal dan program akan mengoutputkan “Setor tunai Rp (jumlah nominal) Berhasil, saldo saat ini Rp(jumlah saldo).
- Method cek informasi. Itu mengoutputkan Info Rekening yang dimiliki Sistem mengeluarkan Nomor Rekening, Nama Pemilik, dan Juga Saldo akhir setelah dilakukannya setor tunai.

2. Membuat Class Main

Agar dapat menjalankan Class rekening tadi diperlukan Class Main. Berikut Screenshot lengkapnya.





Penjelasan :

1) Import Java.util.scanner;

```
import java.util.Scanner;
```

Digunakan untuk mengimpor class Scanner yang berfungsi menerima input dari pengguna melalui keyboard.

2) Deklarasi Class Main

```
public class Main {
    public static void main(String[]args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null; //null karna objek belum di inisialisasi
        boolean isRunning = true;
```

- **public static void main(String[]args) {** adalah method utama yang pertama kali dijalankan ketika program Java dijalankan.
- **Scanner input = new Scanner(System.*in*);** adalah deklarasi dan inisialisasi objek Scanner.
- **Rekening akunAktif = null;** adalah deklarasi variabel referensi objek Rekening, yang awalnya bernilai null (objek belum di insialisasikan)
- **boolean isRunning = true;** adalah deklarasi dan inisialisasi variabel boolean untuk mengontrol program.

3) Menampilkan Judul Program

```
System.out.println("=== SISTEM PERBANKAN MINI ===");
```

Dari sini akan muncul judul dari program yang sedang kita buat.

4) Perulangan Menu utama (while)

```
while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1.Buka Rekening Baru");
    System.out.println("2.Setor Tunai");
    System.out.println("3.Tarik tunai");
    System.out.println("4.Cek Informasi Rekening");
    System.out.println("0.Keluar");
    System.out.println("Pilih Menu: ");
```

- **while (isRunning) {** Digunakan untuk menjalankan menu utama secara berulang selama kondisi isRunning bernilai true,

dan akan berhenti atau keluar saat memilih 0 karna itu menguah nilai isRunning menjadi false.

- Setiap **System.out.println** menjelaskan apa saja menu yang akan muncul contoh :
  1. Buka Rekening, (Pilihan untuk membuka rekening)
  2. Setor tunai (pilihan untuk menyetor saldo)
  3. Tarik tunai (pilihan untuk menarik saldo)
  4, Cek Informasi (pilihan untuk melihat intormasi rekening)
  0. Keluar (pilihan untuk akhiri program)

  Pilih Menu (Meminta user memasukan pemilihan menu)

5) Menerima pilihan user

```
int pilihan = input.nextInt();
input.nextLine(); //Membersihkan buffer enter
```

- **int pilihan = input.nextInt();** digunakan untuk menerima input berupa bilangan bulat dari user dan menyimpannya dalam variable pilihan
- **input.nextLine();** untuk membersihkan buffer enter

6) Menentukan menu pilihan user (switch-case)

```
        switch(pilihan) {
        case 1:
            System.out.print("Masukan No Rekening: ");
            String no = input.nextLine();
            System.out.print("Masukan Nama Pemilik: ");
            String nama = input.nextLine();
            System.out.print("Masukan Saldo Awal: ");
            double saldo = input.nextDouble();

            //Instansiasi Object /Menjalankan Constructor
            akunAktif = new Rekening(no, nama, saldo);
            break;

        case 2:
            if (akunAktif == null) {
                System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");
            } else {
                System.out.print("Masukan nominal setor: ");
                double setor = input.nextDouble();
                akunAktif.setorTunai(setor); //Memanggil Method
            }
            break;

        case 3:

                System.out.println("Fitur ini akan kerjakan sebagai tugas mandiri");

            break;


        case 4:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka rekening!");
            } else {
                akunAktif.cekInformasi();
            }
            break;

        case 0:
            isRunning = false;
            System.out.println("Sistem ditutup. Terima kasih!");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
        }
    }
    input.close();
}
```

- **switch(pilihan) {** digunakan untuk memilih dan menjalankan salah satu blok program berdasarkan pilihan user. Pilihan ini berdasarkan dari menu utama yang sudah di deklarasikan pada class Rekening sebelumnya.
- **Case 1** Bagian User membuka rekening
  Berisikan data yang diminta Masuka No rekening, Masukan Nama Pemilik, Masukan saldo awal. Setelah 3 Data diinput maka akun akan aktif dan baru bisa dilanjut ke Case selanjutnya
- **Case 2** Bagian Setor Tunai
  Bagian ini di gunakan agar user dapat melakukan setor tunai.
  - jika user belum memiliki akun yang aktif maka program akan mengeluarkan error karna akun belum terdeteksi. Dan akan dilakukan perulangan dan user harus melakukan case 1 terlebih dahulu.
  - Jika user sudah memiliki akun aktif maka user dapat melakukan setor tunai berdasarkan yang sudah di deklarasi pada class Rekening
- **Case 3** (Penugasan Mandiri)
  Bagian ini di jadikan penugasan pribadi jadi untuk sekarang informasi yang di hasilkan jika menginputkan Case 3, Output yang akan keluar hanya "Fitur ini akan dikerjakan sebagai tugas mandiri"
- **Case 4** Bagian Cek informasi rekening
  Digunakan untuk menampilkan Informasi rekening
  - jika user belum memiliki akun yang aktif maka program akan mengeluarkan error karna akun belum terdeteksi. Dan akan dilakukan perulangan dan user harus melakukan case 1 terlebih dahulu.
  - Jika user sudah memiliki akun aktif maka user dapat melihat informasi terkait rekening mereka.
- **Case 0** Keluar
  Digunakan untuk mengakhiri program. Nilai yang di hitung **isRunning = false**; membuat kondisi while jadi false dan perulangan berhenti.
- **Default**
  Digunakan apanila user memasukan menu yang tidak ada.
- **input.close();** Digunakan untuk menutup objek scanner setelah input dari user selesai.

2.3 Latihan/Tugas

1. **Implementasi Fitur Tarik Tunai**
    - Tambahkan method **tarikTunai(double nominal)** ke class rekening. Dengan aturan
        a. Saldo tidak boleh minus dan minimal Tarik tunai adalah 10.000
        b. Jika user menarik uang lebih dari saldo saat ini atau kecil dari 10.000 maka akan menampilkan "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau “Transaksi Gagal : Minimal nomonal penarikan 10.000”.
        c. Terapkan method pada menu nomor 3 pada class main.

Method tarikTunai(double nominal) pada class rekening

```
public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Transaksi Gagal: Minimal Penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal saldo tidak mencukupi.Saldo Anda: Rp" + saldo);
        }
        else {
            saldo -= nominal;
            System.out.println("Transaksi Berhasil");
        }

}
```

Penjelasan:

Pada Method Tarik tunai ini di sebutkan

- Jika nominal penarikan kecil dari 10.000 maka system akan mengoutputkan "Transaksi Gagal: Minimal Penarikan 10.000"
- Jika nominal penarikan besar dari saldo user maka system akan mengoutputkan "Transaksi Gagal saldo tidak mencukupi.Saldo Anda: Rp"
- Namun jika selain itu, seperti saldo kecil sama dengan nominal dan syarat lainnya terpenuhi maka system akan mengoutputkan “Transaksi Berhasil”

Selanjutnya, tambahkan method tarikTunai ini pada Class main di bagian Switch case 3

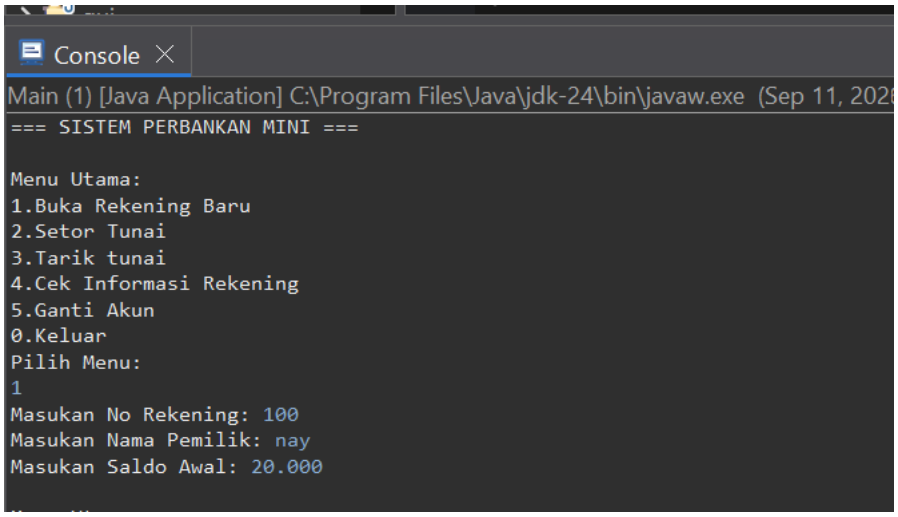
```
    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");
        } else {
            System.out.print("Masukan nominal tarik: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik); //Memanggil Method
        }
        break;
```

Penjelasan:

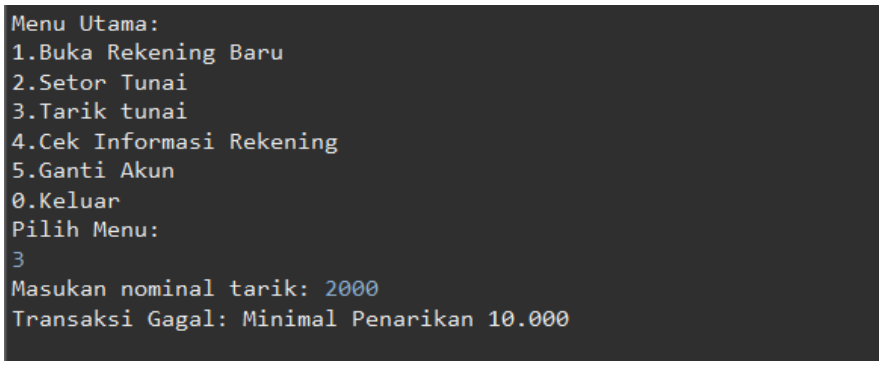
Pada Case 3

- Jika akun aktif nya tidak ada/ null maka program akan mengoutputkna "Error: Mohon Maaf, Anda belum memiliki nomor rekening!"
- Jika akun sudah ada, program akan mengoutputkan "Masukan Nominal Tarik:" dan system akan memproses dan memanggil method yang sudah di deklarasikan pada class rekening tadi.

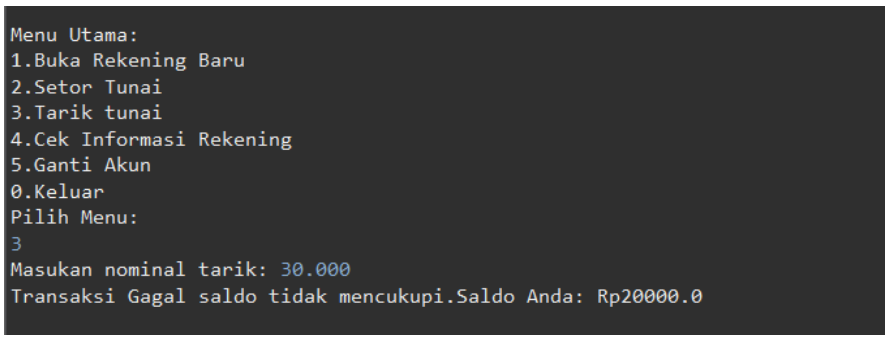
OUTPUT



(Buka Rekening baru)



(Transaksi Gagal Minimal penarikan !0.000)



(Transaksi gagal saldo tidak mencukupi)

```
Masukan nominal tarik: 15.000
Transaksi Berhasil

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
```

(Transaksi Berhasil)

2. Tantangan Multi-Akun

Membuat multi akun dengan intruksi

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection (Gunakan ArrayList<Rekening>)
- Ubah logika menu "Buka Rekening". Setia pada user yang membuka rekening baru objek itu tambahkan ke arraylist.
- Tambahkan menu baru Ganti akun, dimana system meminta nomor rekening kemudian mencari objek yang cocok pada arraylist untuk dijadikan akun aktif

ArrayList digunakan untuk menyimpan beberapa objek Rekening dalam satu tempat. Dengan demikian, setiap rekening yang dibuat oleh pengguna dapat disimpan dan tetap dapat diakses kembali tanpa menghapus rekening sebelumnya. Pada Class Main :

1) Tambahkan **import java.util ArrayList**, Digunakan untuk mengimpor class Scanner yang berfungsi menerima input dari pengguna melalui keyboard.

```
import java.util.ArrayList;
```

2) Deklarasikan pada public class main **ArrayList<Rekening> daftarRekening = new ArrayList<>();**

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();
```

Digunakan untuk menyimpan data data rekening dan daftar rekening sebagai tempat penyimpanan nya.

3) Instialisasi objek pada **case 1** untuk menjalankan Constructor

```
    //Instansiasi Object /Menjalankan Constructor
    Rekening rekeningBaru = new Rekening(no, nama, saldo);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
break;
```

- **Rekening rekeningBaru = new Rekening(no, nama, saldo);**
  Diminta user meng inputkan rekening baru terlebih dahulu
- **daftarRekening.add(rekeningBaru);**
  Kemudian rekening akan masuk ke daftar rekening ini
- **akunAktif = rekeningBaru;**
  Akun yang aktif adalah akun yang baru di buat

4) Fitur Ganti Akun
   a. Pada bagian while atau perulangan tambahkan **System.*out*.println("5.Ganti Akun");**
   b. Buat **Case 5** Bagian untuk Ganti akun

```
case 5:
    System.out.print("Masukan No Rekening: ");
    String cariNo = input.nextLine();
    boolean ditemukan = false;

    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(cariNo)) {
            akunAktif = rekening;
            ditemukan = true;
            System.out.println("Akun Berhasil Diganti.");
            break;
        }
    }

    if (!ditemukan) {
        System.out.println("Rekening tidak ditemukan.");
    }
    break;
```

- **System.out.print("Masukan No Rekening: ");**
  String cariNo = input.nextLine();
  Digunakan untuk meminta nomor rekening User
- **boolean ditemukan = false;** Digunakan sebagai variable penanda apakah rekening yang dicari sudah di temukan atau belum, nilai awal fase karna rekening belum ditemukan.
- **for (Rekening rekening : daftarRekening) {** Bagian yang digunakan untuk mengcek objek rekening yang tersimpan di daftar rekening
- **if (rekening.getNomorRekening().equals(cariNo)) {** Bagian untuk membandingkan nomor rekening yang tersimpan dengan rekening yang dimasukan user
- **akunAktif = rekening;** Jika rekening ditemukan maka akan jadi akun aktif

- **ditemukan = true;** Mengubah status false diawal menjadi true karna rekening sudah ditemukan.
- **System.out.println("Akun Berhasil Diganti.");** Menampilkan output bahwa pergantian akun sudah berhasil
- **break;** digunakan untuk menghentikan perulangan for setelah rekening ditemukan.
- **if (!ditemukan) {**
  System.out.println("Rekening tidak ditemukan."); Jika memang tidak ditemukan program akan mengoutputkan "Rekening tidak ditemukan"

c. Masukan deklarasi Constructor baru pada class Rekening

```
public Rekening(String nomorRekening, String namaPemilik, double saldo) {
        this.nomorRekening = nomorRekening;
        this.namaPemilik = namaPemilik;
        this.saldo = saldo;
    }
```

Constructor ini digunakan untuk memberi nilai awal pada objek rekening, agar dapat dijalan kan pada bagian Class Main yang sudah di tembahkan tadi

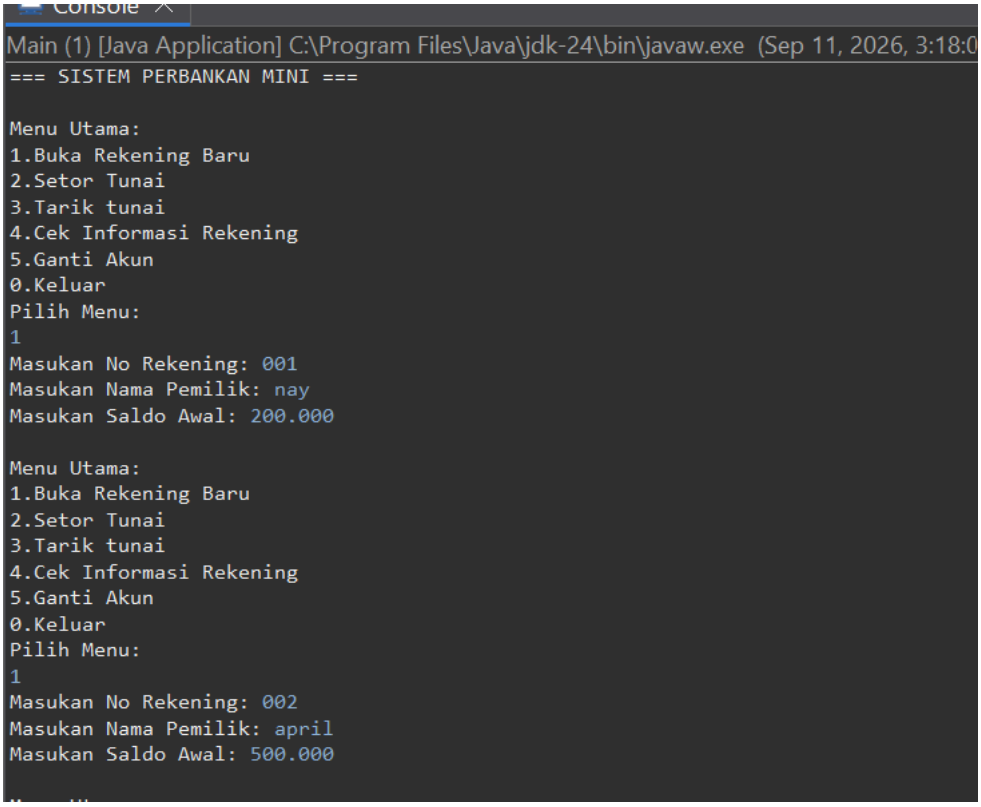
d. Tambahkan Method getter pada Class Rekening

```
    public String getNomorRekening() {
        return nomorRekening;
    }
```

Method ini gunakan untuk mengambil atau mendapatkan nilai nomor rekening pada objek reening. Getter disini digunakan pada fitur Ganti Akun

If (rekening.getNomorRekening().equals(cariNo)) { yang mengartikan program ingin mengambil semua objek dalam daftar rekening

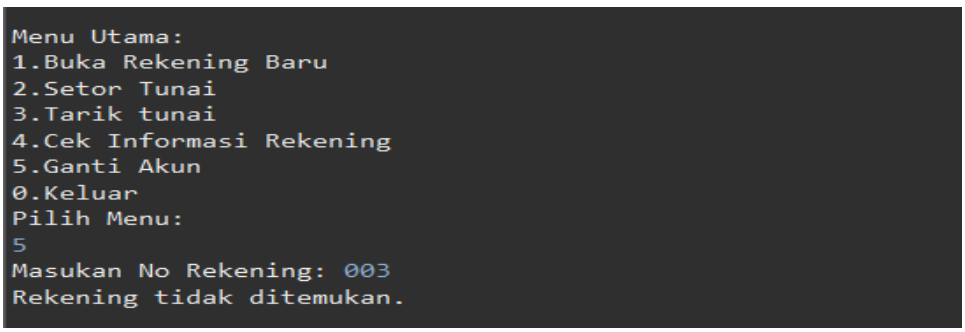
OUTPUT



(Membuat banyak Rekening dan dapat tersimpan di daftar rekening tanpa terhapus)



(Mengganti akun berhasil dilakukan,karna nomor rekening ditemukan)



(Gagalk arna rekening tidak ditemukan)

LAPORAN PRAKTIKUM PEKAN 2

PEMOGRAMAN BERORIENTASI OBJEK

Relasi Antar Objek (Class Diagram & Asosiasi)

Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah,S.ST., M.Kom.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 2 mata kuliah Pemrograman Berorientasi Objek dengan materi Relasi Antar Objek (Class Diagram & Asosiasi) dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Melalui praktikum ini, penulis mempelajari konsep dasar pemrograman berorientasi objek, khususnya mengenai Relasi Antar Objek (Class Diagram & Asosiasi), serta penerapannya dalam membangun program sederhana berupa Transaksi Debit dan Kredit. Praktikum ini juga membantu penulis memahami bagaimana objek dapat saling berinteraksi dan bagaimana state suatu objek dapat dimanipulasi melalui method.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dalam penyusunan maupun penyajian materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan evaluasi dan perbaikan untuk laporan maupun praktikum selanjutnya.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai konsep dasar Pemrograman Berorientasi Objek, khususnya dalam Relasi Antar Objek (Class Diagram & Asosiasi).

Padang, 14 September 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan paradigma pemrograman yang menggunakan objek sebagai dasar dalam membangun suatu program. Setelah memahami konsep dasar seperti class, object, constructor, dan method, pemahaman mengenai hubungan atau relasi antarobjek menjadi salah satu hal penting dalam pengembangan program berbasis objek. Relasi antarobjek memungkinkan beberapa objek dalam suatu sistem untuk saling berhubungan dan berinteraksi sesuai dengan fungsi masing-masing.

Salah satu cara untuk menggambarkan relasi antarobjek adalah menggunakan Class Diagram. Class Diagram merupakan bagian dari Unified Modeling Language (UML) yang digunakan untuk menggambarkan struktur suatu sistem, seperti class, atribut, method, serta hubungan yang terdapat antarclass. Dengan adanya Class Diagram, struktur dan hubungan antarobjek dalam suatu program dapat dirancang dan dipahami dengan lebih jelas sebelum program diimplementasikan.

Salah satu bentuk relasi yang umum digunakan dalam Pemrograman Berorientasi Objek adalah asosiasi. Asosiasi menunjukkan adanya hubungan antara satu class dengan class lainnya yang memungkinkan objek dari kedua class tersebut saling berinteraksi. Penerapan asosiasi dapat ditemukan dalam berbagai sistem, salah satunya pada sistem perbankan yang memiliki hubungan antara data rekening dengan transaksi yang dilakukan.

Pada praktikum ini, konsep Relasi Antar Objek diterapkan melalui pembuatan program sederhana berupa Transaksi Debit dan Kredit. Melalui program tersebut, mahasiswa dapat memahami bagaimana beberapa class saling berhubungan, bagaimana objek berinteraksi melalui method, serta bagaimana perubahan state pada suatu objek dapat terjadi akibat interaksi dengan objek lainnya.

Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep Relasi Antar Objek, khususnya Class Diagram dan Asosiasi, serta mampu merancang dan mengimplementasikan hubungan antarclass dalam bahasa pemrograman Java melalui program sederhana Transaksi Debit dan Kredit.

1.2 Tujuan

1. Membaca dan menerjemahkan Class Diagram ke dalam kode program java.
2. Memahami dan mengimplementasika relasi Asosiasi (one to many)
3. Memanajemen kolelksi objek di dalam objek lain menggunakan tipe data bentukan dan struktur dinamis (ArrayList)

1.3 Manfaat

1. Menambah pemahaman mengenai cara membaca dan menerjemahkan Class Diagram ke dalam kode program Java.
2. Meningkatkan kemampuan dalam memahami dan menerapkan relasi Asosiasi (one to many) antarobjek dalam Pemrograman Berorientasi Objek.
3. Memahami cara mengelola dan memanajemen kumpulan objek di dalam objek lain menggunakan tipe data bentukan dan struktur dinamis ArrayList.
4. Membantu mahasiswa dalam memahami hubungan dan interaksi antarobjek sehingga dapat membuat program yang lebih terstruktur dan mudah dikembangkan.
5. Meningkatkan kemampuan mahasiswa dalam menerapkan konsep Pemrograman Berorientasi Objek melalui pembuatan program sederhana menggunakan bahasa pemrograman Java

BAB II

PEMBAHASAN

2.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), sebuah program tidak hanya terdiri dari satu objek, tetapi dapat memiliki banyak objek yang saling berhubungan dan berinteraksi. Hubungan antarobjek tersebut digunakan untuk menggambarkan bagaimana suatu objek dapat berkomunikasi atau bekerja sama dengan objek lainnya. Untuk mempermudah perancangan hubungan tersebut, digunakan Class Diagram dan konsep asosiasi.

• **Class Diagram**, merupakan salah satu diagram dalam Unified Modeling Language (UML) yang digunakan untuk menggambarkan struktur suatu sistem. Class Diagram dapat menunjukkan class yang terdapat dalam program beserta atribut, method, dan hubungan antarclass. Dengan adanya Class Diagram, struktur program dapat dirancang terlebih dahulu sehingga proses implementasi ke dalam kode Java menjadi lebih terarah.

• **Relasi Antar Objek**, merupakan hubungan yang terjadi antara objek dari suatu class dengan objek dari class lainnya. Relasi ini memungkinkan objek untuk saling berinteraksi dalam menjalankan fungsi tertentu. Dalam sebuah sistem, relasi antarobjek diperlukan agar setiap objek dapat menjalankan perannya dan bekerja sama untuk menghasilkan suatu proses.

• **Asosiasi**, merupakan salah satu bentuk relasi antarclass yang menunjukkan adanya hubungan antara satu class dengan class lainnya. Asosiasi dapat digunakan ketika suatu objek membutuhkan atau memiliki hubungan dengan objek lain. Salah satu bentuk asosiasi adalah **one to many (satu ke banyak)**, yaitu ketika satu objek dari suatu class dapat memiliki hubungan dengan banyak objek dari class lainnya.

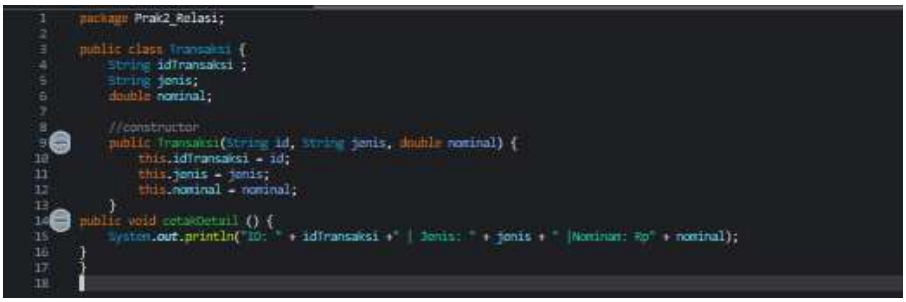
• **Asosiasi One to Many**, dapat diterapkan pada sistem transaksi perbankan. Sebagai contoh, satu objek **Rekening** dapat memiliki banyak objek **Transaksi**. Satu rekening dapat digunakan untuk melakukan beberapa transaksi, baik transaksi debit maupun kredit. Hubungan tersebut dapat digambarkan dalam Class Diagram dan kemudian diterapkan ke dalam kode program Java.

• **ArrayList**, merupakan salah satu struktur data dinamis dalam Java yang digunakan untuk menyimpan sekumpulan objek. ArrayList memiliki ukuran yang dapat bertambah atau berkurang sesuai dengan jumlah data yang disimpan. Dalam penerapan asosiasi one to many, ArrayList dapat digunakan untuk menyimpan banyak objek dari suatu class di dalam objek dari class lainnya. Contohnya, objek Rekening dapat memiliki sebuah ArrayList yang digunakan untuk menyimpan kumpulan objek Transaksi.

Dalam konteks program **Transaksi Debit dan Kredit**, class Rekening dapat digunakan untuk menyimpan informasi rekening, sedangkan class Transaksi digunakan untuk menyimpan informasi transaksi yang dilakukan. Hubungan antara keduanya menggunakan asosiasi one to many, karena satu rekening dapat memiliki banyak transaksi. Kumpulan transaksi tersebut dapat dikelola menggunakan ArrayList sehingga data transaksi dapat ditambahkan dan dikelola secara dinamis.

2.2 Langkah Langkah

1. **Membuat Class Transaksi**
   1) Pertama saya membuat Package "Prak2_Relasi"
   2) Selanjutnya dilanjukan dengan membuat Class Transaksi, Berikut Screenshot lengkap nya



```
package Prak2_Relasi;

public class Transaksi {
    String idTransaksi ;
    String jenis;
    double nominal;

    //constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
public void cetakDetail () {
    System.out.println("ID: " + idTransaksi +" | Jenis: " + jenis + " |Nominal: Rp" + nominal);
}
}
```

Penjelasan:

1) Buat class Transaksi

```
public class Transaksi {
```

Ini mengartikan kita sedang membuat sebuah class bernama Transaksi. Class bisa dianggap sebagai cetakan/model untuk membuat objek Transaksi.

2) Membuat State/Atribut

```
public class Transaksi {
    String idTransaksi ;
    String jenis;
    double nominal;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class transaksi ini tersimpan data berupa

- idTransaksi dengan tipe data string
- jenis dengan tipe data string
- nominal dengan tipe data double.

3) Buat Constructor

```
//constructor
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek Transaksi Ketika objek dibuat.

- idTransaksi di inisialisasikan dengan nilai id
- jenis diinisialisasikan dengan nilai jenis transaksi, yaitu jenis transaksi debit atau kredit
- nominal diinisialisasikan dengan nilai nominak transaksi

4) Buat Method/Behavior

```
public void cetakDetail () {
    System.out.println("ID: " + idTransaksi +" | Jenis: " + jenis + " |Nominal: Rp" + nominal);
}
}
```

Method cetakDetail() digunakan untuk menampilkan detail transaksi yang berisi ID transaksi, jenis transaksi, dan nominal transaksi. Program akan menampilkan informasi dengan format “ID: (ID transaksi) | Jenis: (jenis transaksi) | Nominal: Rp (nominal)”.

2. Membuat Relasi pada Class Rekening

Salin Code Program yang sudah dibuat pada Prak1_MiniBank, sebagai relasi nya. Dan akan ditambahkan atribut baru yang mempresentasikan relasi.

```
1  package Prak2_Relasi;
2  import java.util.ArrayList;
3
4  public class Rekening {
5      String nomorRekening;
6      String namaPemilik;
7      double saldo;
8
9      //Implementasikan Asosiasi (1-to-many)
10     ArrayList<Transaksi> riwayatTransaksi;
11
12     public Rekening(String nomorRekening, String namaPemilik, double saldoAwal) {
13         this.nomorRekening = nomorRekening;
14         this.namaPemilik = namaPemilik;
15         this.saldo = saldoAwal;
16
17
18         //wajib menginisialisasikan Arraylist di dalam constructor agar tidak NullPointerException
19         this.riwayatTransaksi = new ArrayList<>();
20
21         System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
22     }
23
```

- **ArrayList<Transaksi> riwayatTransaksi;**

Digunakan untuk menyimpan kumpulan data transaksi yang dilakukan oleh rekening. Jadi, setiap transaksi bisa dicatat dan disimpan dalam riwayatTransaksi.

- **this.riwayatTransaksi = new ArrayList<>();**
  Digunakan untuk membuat ArrayList baru yang kosong sebagai tempat menyimpan riwayat transaksi. this menunjukkan bahwa yang digunakan adalah atribut riwayatTransaksi milik objek tersebut.
- **System.*out*.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");**
  Digunakan untuk menampilkan pesan ke layar bahwa rekening berhasil dibuat, sekaligus menampilkan nama pemilik rekening.

3. **Membuat Riwayat transaksi**
   Perbaiki method setor tunai dan Tarik tunai di class rekening java aga rotomatis menginstansiasi obejk transaksi dan menyimpannya ke riwayatTransaksi saat transaksi berhasil.

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        //Mencatat riwayat (Membuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil Saldo saat ini: Rp" + saldo);
    }else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0! ");
    }
}
```

Method Setor tunai menjelaskan bahwa jika nominal lebih dari 0, maka saldo akan bertambah sesuai nominal yang disetor, kemudian transaksi dicatat ke dalam riwayat transaksi dan program akan menampilkan “Setor tunai Rp (jumlah nominal) berhasil, saldo saat ini Rp (jumlah saldo)”. Jika nominal 0 atau kurang, maka program akan menampilkan pesan bahwa setor tunai gagal.

- **String idTrx = "TRX-S-" + System.currentTimeMillis();** ini digunakan untuk membuat ID transakasi otomatis. **TRX-S-** menandakan transaksi setor, sedangkan **currentTimeMillis()** memberikan angka waktu saat transaksi dibuat.
- **Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);** Membuat objek transaksi baru dengan **idTrx**(id Transaksi), **“Kredit”**(Jenis transaksi setor), **nominal**(jumlah uang)
- **riwayatTransaksi.add(trxBaru);** Memasukkan transaksi tersebut ke dalam riwayat transaksi.

Latihan/Tugas

1. **Method Tarik Tunai**

```
public void tarikTunai(double nominal) {
    String idTrx = "TRX-T-" + System.currentTimeMillis();
    Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
    riwayatTransaksi.add(trxBaru);

        if (nominal < 10000) {
            System.out.println("Transaksi Gagal: Minimal Penarikan 10.000");
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal saldo tidak mencukupi.Saldo Anda: Rp" + saldo);
        }
        else {
            saldo -= nominal;
            System.out.println("Transaksi Berhasil");
        }
```

Pada Class Rekening, pada bagian method Tarik tunai ubah beberapa bagian seperti syarat dari tugas yaitu Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

- **String idTrx = "TRX-T-" + System.*currentTimeMillis*();** ini digunakan untuk membuat ID transakasi otomatis. **TRX-T** menandakan transaksi tarik, sedangkan **currentTimeMillis()** memberikan angka waktu saat transaksi dibuat.
- **Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);** Membuat objek transaksi baru dengan **idTrx**(id Transaksi), **“Debit”**(Jenis transaksi setor), **nominal**(jumlah uang)
- **riwayatTransaksi.add(trxBaru)** Memasukkan transaksi tersebut ke dalam riwayat transaksi.

2. **Cetak Mutasi Rekening**

1) Tambahkan Method cetakMutasi pada bagian Class Rekening

```
public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }
    }
```

Memnambahkan perulangan for-each untuk menulusuri koleksi riwayatTransaksi nya. Jika riwayatTransaksi kosong maka system akan print “Belum ada transaksi pada rekening ini”. Selain itu utntuk transaksi akan diperikasa Riwayat dan cetak detail nya.

2) Pada Class Main tambahkan while dan juga case 6

```
    while (isRunning) {
        System.out.println("\nMenu Utama:");
        System.out.println("1.Buka Rekening Baru");
        System.out.println("2.Setor Tunai");
        System.out.println("3.Tarik tunai");
        System.out.println("4.Cek Informasi Rekening");
        System.out.println("5.Ganti Akun");
        System.out.println("6.Cetak Mutasi (Riwayat)");
        System.out.println("0.Keluar");
        System.out.println("Pilih Menu: ");
```

Tambahkan pada bagian 6.Cetak Mutasi (Riwayat)

```
case 6:
    akunAktif.cetakMutasi();
    break;
```

Menambahkan case ke 6 yang digunakan untuk menjalankan method cetakMutasi() pada akun yang sedang aktif untuk menampilkan Riwayat transaksi yang telah dilakukan dan break untuk keluar dari case tersebut.

OUTPUT

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: Nay
Masukan Saldo Awal: 500.000
Rekening atas nama Nay berhasil dibuat.

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
2
Masukan nominal setor: 50.000
Setor tunai Rp50000.0 berhasil Saldo saat ini: Rp350000.0
```

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
3
Masukan nominal tarik: 150.000
Transaksi Berhasil

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
6
ID: TRX-S-1789828151753 | Jenis: Kredit |Nominal: Rp50000.0
ID: TRX-T-1789828186817 | Jenis: Debit |Nominal: Rp150000.0
```

Buka rekening baru dan lanjutkan dengan setor tunai dan Tarik tunai. Dan lanjutkan dengan cetak mutasi atau Riwayat.

LAPORAN PRAKTIKUM PEKAN 3

PEMOGRAMAN BERORIENTASI OBJEK

Keamanan & Integritas Data (Enkapsulasi)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah,S.ST., M.Kom.

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam Pemrograman Berorientasi Objek (PBO), keamanan dan integritas data merupakan hal yang penting dalam pengembangan sebuah program. Data yang terdapat di dalam suatu class sebaiknya tidak dapat diakses atau dimanipulasi secara langsung dari luar class karena dapat menyebabkan perubahan data yang tidak sesuai dengan aturan program. Oleh karena itu, diperlukan suatu mekanisme untuk membatasi akses terhadap data tersebut.

Salah satu konsep yang digunakan adalah enkapsulasi dan Information Hiding. Enkapsulasi memungkinkan atribut dalam suatu class dilindungi menggunakan access modifier seperti private, protected, dan public. Selain itu, getter dan setter dapat digunakan untuk mengatur proses pembacaan dan perubahan data.

Pada praktikum ini, konsep keamanan data diterapkan pada program Mini Bank dengan mengamankan atribut pada class Transaksi dan Rekening. Selain itu, ditambahkan PIN sebagai sistem autentikasi yang digunakan sebelum melakukan tarik tunai dan melihat riwayat transaksi.

1.2 Tujuan

1. Memahami konsep Information Hiding dalam Pemrograman Berorientasi Objek.
2. Memahami penggunaan access modifier private, protected, dan public.
3. Menerapkan konsep enkapsulasi menggunakan getter dan setter.
4. Mengamankan data pada class Transaksi dan Rekening.
5. Menerapkan autentikasi PIN pada program Mini Bank.

1.3 Manfaat

1. Memahami pentingnya keamanan dan integritas data dalam program.
2. Mampu menerapkan access modifier untuk membatasi akses terhadap atribut.
3. Mampu menerapkan enkapsulasi pada program Java.
4. Memahami penggunaan getter, setter, dan autentikasi PIN.
5. Meningkatkan kemampuan dalam membuat program yang memiliki keamanan data lebih baik.

BAB II

PEMBAHASAN

2.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), keamanan data diperlukan agar atribut dalam suatu class tidak dapat dimanipulasi secara langsung dari luar class. Konsep yang digunakan untuk membatasi akses tersebut adalah **Information Hiding** dan **enkapsulasi**.

• **Information Hiding**, merupakan konsep untuk menyembunyikan detail implementasi internal suatu object sehingga data tidak dapat diakses secara langsung. Object lebih mengutamakan penggunaan method sebagai behavior daripada memberikan akses langsung terhadap variabel.

• **Access Modifier**, digunakan untuk mengatur tingkat akses atribut dan method. private hanya dapat diakses dalam class, protected dapat diakses dalam package dan subclass, sedangkan public dapat diakses dari mana saja.

• **Getter dan Setter**, digunakan sebagai perantara untuk mengakses atribut yang bersifat private. Getter digunakan untuk membaca data, sedangkan setter digunakan untuk mengubah data.

• **Enkapsulasi**, merupakan konsep untuk membatasi akses langsung terhadap data dalam class dengan melindungi atribut menggunakan access modifier dan menyediakan method yang diperlukan untuk mengakses data tersebut.

• **Autentikasi PIN**, digunakan untuk memeriksa PIN yang dimasukkan pengguna sebelum menjalankan fungsi tertentu. Pada praktikum ini, autentikasi digunakan pada proses tarik tunai dan cetak mutasi. Jika PIN benar, proses dilanjutkan, sedangkan jika salah maka akses ditolak.

2.2 Langkah Langkah

1. Membuat Class Transaksi

1) Pertama saya membuat Package “prak3_enkapsulasi”
2) Selanjutnya dilanjukan dengan membuat Class Transaksi, Berikut Screenshot lengkap nya

```java
package prak3_enkapsulasi;

public class Transaksi {
    //mengubah atribut id private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
    // Hanya nyediakan getter (Read Only)
    public String getIdTransaksi() { return idTransaksi;}
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal;}

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis:"
        + jenis + " | Nominal: Rp" + nominal);
    }

}
```

Penjelasan:

1) Buat class Transaksi

```java
public class Transaksi {
```

Ini mengartikan kita sedang membuat sebuah class bernama Transaksi. Class bisa dianggap sebagai cetakan/model untuk membuat objek Transaksi.

2) Membuat State/Atribut

```java
//mengubah atribut id private
private String idTransaksi;
private String jenis;
private double nominal;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class transaksi ini tersimpan data berupa

- idTransaksi dengan tipe data string

- jenis dengan tipe data string
- nominal dengan tipe data double.

Kata private digunakan agar atribut hanya dapat diakses secara langsung dari dalam class Transaksi. Hal ini merupakan penerapan konsep **enkapsulasi**.

3) Buat Constructor

```
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek Transaksi Ketika objek dibuat.

- idTransaksi di inisialisasikan dengan nilai id
- jenis diinisialisasikan dengan nilai jenis transaksi, yaitu jenis transaksi debit atau kredit
- nominal diinisialisasikan dengan nilai nominak transaksi

this digunakan untuk menunjukkan atribut yang berada di dalam class Transaksi.

4) Membuat Method Getter

```
// Hanya nyediakan getter (Read Only)
public String getIdTransaksi() { return idTransaksi;}
public String getJenis() { return jenis; }
public double getNominal() { return nominal;}
```

Method getter digunakan untuk mengambil atau membaca nilai atribut yang bersifat private.Pada class Transaksi terdapat tiga getter, yaitu:

- getIdTransaksi() digunakan untuk mengambil nilai idTransaksi.
- getJenis() digunakan untuk mengambil nilai jenis.
- getNominal() digunakan untuk mengambil nilai nominal.

Getter digunakan karena atribut dibuat private, sehingga nilai atribut tidak dapat diakses secara langsung dari luar class

5) Buat Method cetakDetail

```
    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis:"
        + jenis + " | Nominal: Rp" + nominal);
    }

}
```

Method cetakDetail() digunakan untuk menampilkan detail transaksi yang berisi ID transaksi, jenis transaksi, dan nominal transaksi. Program akan menampilkan informasi dengan format “ID: (ID transaksi) | Jenis: (jenis transaksi) | Nominal: Rp (nominal)”.

2. Membuat Class Rekening

3) Salin Code Program yang sudah dibuat pada praktikum prak2_Relasi, Dan ubah atau tambah beberapa bagian saja. Berikut Screenshot lengkap nya

```
1  package prak3_enkapsulasi;
2  import java.util.ArrayList;
3
4  public class Rekening {
5      // Mengunci atribut dengan private
6      private String nomorRekening;
7      private String namaPemilik;
8      private double saldo;
9      private String pin; //Data Sensitif
10
11     private ArrayList<Transaksi> riwayatTransaksi;
12
13     //2.Modifikasi Consstructor untuk menecima PIN awal
14     public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
15         this.nomorRekening = nomor;
16         this.namaPemilik = nama;
17         this.saldo = saldoAwal;
18
19         //validasi PIN dalam Constructor
20         if (pinAwal.length() == 6) {
21             this.pin =pinAwal;
22         } else {
23         System.out.println("Peringatan: PIN harus 6 digit Menggunakan PIN default 123456");
24         this.pin = "123456";
25         }
26     this.riwayatTransaksi = new ArrayList<>();
27     System.out.println("Rekening atas nama " +  namaPemilik  + " berhasil dibuat.");
28 }
29     //3. Getter untuk atribut yang diizinkan dibaca publik
30     public String getNomorRekening() { return nomorRekening; }
31     public String getNamaPemilik() { return namaPemilik; }
32
33     //4. Method Otentikasi Internal (Validasi Enkapsulasi)
34     public boolean otentikasi(String inputPin) {
35         return this.pin.equals(inputPin);
36     }
```

Pada bagian ini yang di ubah dan ada tambahan, berikut penjelasannya:

1) Mengunci Atribut dengan Private

```
public class Rekening {
    // Mengunci atribut dengan private
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; //Data Sensitif

    private ArrayList<Transaksi> riwayatTransaksi;
```

Atribut merupakan variabel yang digunakan untuk menyimpan informasi dari suatu objek. Pada class Rekening terdapat beberapa atribut, yaitu:

- nomorRekening dengan tipe data **String**, digunakan untuk menyimpan nomor rekening.
- namaPemilik dengan tipe data **String**, digunakan untuk menyimpan nama pemilik rekening
- saldo dengan tipe data **double**, digunakan untuk menyimpan jumlah saldo rekening.
- pin dengan tipe data **String**, digunakan untuk menyimpan PIN rekening. PIN termasuk data sensitif.
- riwayatTransaksi dengan tipe **ArrayList<Transaksi>**, digunakan untuk menyimpan daftar transaksi yang dilakukan pada rekening.

Semua atribut menggunakan private agar data tidak dapat diakses atau diubah secara langsung dari luar class. Hal ini merupakan penerapan konsep **enkapsulasi**.

2) Modifikasi Constructor untuk menerima PIN awal

```
//2.Modifikasi Consstructor untuk menerima PIN awal
public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;
```

Constructor digunakan untuk memberikan nilai awal pada objek Rekening ketika objek tersebut dibuat.

- nomorRekening diinisialisasikan dengan nilai dari parameter nomor
- nomorRekening diinisialisasikan dengan nilai dari parameter nomor
- saldo diinisialisasikan dengan nilai dari parameter saldoAwal.

Validasi PIN dalam Constructor

```
        //validasi PIN dalam Constructor
        if (pinAwal.length() == 6) {
            this.pin =pinAwal;
        } else {
        System.out.println("Peringatan: PIN harus 6 digit Menggunakan PIN default 123456");
        this.pin = "123456";
        }
    this.riwayatTransaksi = new ArrayList<>();
    System.out.println("Rekening atas nama " +  namaPemilik  + " berhasil dibuat.");
}
```

Bagian ini digunakan untuk melakukan **validasi PIN** ketika rekening dibuat.

- Jika panjang PIN sama dengan 6 karakter, maka PIN tersebut akan digunakan.
- Jika PIN tidak berjumlah 6 karakter, program akan memberikan peringatan.
- Kemudian program menggunakan PIN default yaitu 123456

Dengan adanya validasi ini, data PIN yang disimpan dapat dikontrol sejak awal pembuatan objek

this.riwayatTransaksi = new ArrayList<>(); Kode ini digunakan untuk membuat sebuah ArrayList baru yang akan digunakan untuk menyimpan objek-objek Transaksi.

3) Getter untuk atribut yang diizinkan dibaca public

```
//3. Getter untuk atribut yang diizinkan dibaca publik
public String getNomorRekening() { return nomorRekening; }
public String getNamaPemilik() { return namaPemilik; }
```

Method getter digunakan untuk **mengambil atau membaca nilai atribut** yang bersifat private.

Pada class Rekening terdapat dua getter:

- getNomorRekening() digunakan untuk mengambil nomor rekening.
- getNamaPemilik() digunakan untuk mengambil nama pemilik rekening.

Tidak dibuat getter untuk pin karena PIN merupakan **data sensitif** dan tidak boleh sembarangan diakses dari luar class.

4) Method Otentikasi internal (Validasi Enkasulasi)

```
//4. Method Otentikasi Internal (Validasi Enkapsulasi)
public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}
```

Method otentikasi() digunakan untuk memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening.

- inputPin merupakan PIN yang dimasukkan oleh pengguna.

- this.pin merupakan PIN yang tersimpan di dalam rekening.
- equals() digunakan untuk membandingkan kedua PIN.
- Method mengembalikan nilai true jika PIN sesuai dan false jika PIN tidak sesuai.

Dengan cara ini, PIN tetap tersimpan secara private dan tidak perlu diberikan langsung kepada class lain.

Pertanyaan

Mengapa ditdak dibuat method setSaldo() dan getPin()?

Jawaban

Method setSaldo() tidak dibuat karena perubahan saldo harus dilakukan melalui method setorTunai() dan tarikTunai() agar tetap melalui proses dan validasi yang telah ditentukan. Sementara itu, method getPin() tidak dibuat karena PIN merupakan data sensitif yang harus dirahasiakan. Sebagai gantinya, digunakan method otentikasi() untuk memeriksa kesesuaian PIN tanpa menampilkan atau memberikan nilai PIN secara langsung.

TUGAS/LATIHAN

Perbarui program Main.java dengan aturan

a. Menu 1 Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).
b. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.
   • Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
   • Jika mengembalikan nilai true, jalankan transaksi.
   • Jika false, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Jawaban:

a. Class Main.java Menu 1 (Buka Rekening)

```
case 1:
    System.out.print("Masukan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();

    // minta input PIN
    System.out.print("Masukan PIN(6 digit):");
    String pin = input.nextLine();

    //Instansiasi Object /Menjalankan Constructor
    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);

    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
break;
```

Penjelasan

- **System.out.print("Masukan No Rekening: ");**
  String no = input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan nomor rekening. Nilai yang dimasukkan oleh pengguna disimpan ke dalam variabel no dengan tipe data String.

- **System.out.print("Masukan Nama Pemilik: ");**
  String nama = input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan nama pemilik rekening. Nama yang dimasukkan akan disimpan ke dalam variabel nama dengan tipe data String.

- **System.out.print("Masukan Saldo Awal: ");**
  double saldo = input.nextDouble();
  input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan saldo awal rekening**.**

  - saldo menggunakan tipe data double karena menyimpan nilai berupa angka.
  - **input.nextDouble()** digunakan untuk membaca input berupa angka
  - **input.nextLine()** digunakan untuk membersihkan sisa enter setelah penggunaan **nextDouble()** agar input berikutnya dapat terbaca dengan benar.

- **System.out.print("Masukan PIN(6 digit):");**

String pin = input.nextLine();

Kode ini digunakan untuk meminta pengguna memasukkan PIN rekening. PIN disimpan dalam variabel pin dengan tipe data String. PIN menggunakan String karena PIN merupakan kode, bukan nilai yang digunakan untuk perhitungan.

- **Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);**
  daftarRekening.add(rekeningBaru);
  akunAktif = rekeningBaru;

Kode tersebut digunakan untuk membuat objek rekening baru, menyimpannya ke dalam daftar rekening, dan menjadikannya sebagai akun aktif.

- **new Rekening(no, nama, saldo, pin)** digunakan untuk membuat objek Rekening baru dengan data nomor rekening, nama pemilik, saldo awal, dan PIN.
- **daftarRekening.add(rekeningBaru)** digunakan untuk menambahkan rekening baru ke dalam daftar rekening.
- **akunAktif = rekeningBaru** digunakan untuk menjadikan rekening yang baru dibuat sebagai akun yang sedang aktif digunakan.

OUTPUT

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: Naysha
Masukan Saldo Awal: 200000
Masukan PIN(6 digit):123456
Rekening atas nama Naysha berhasil dibuat.
```

b. Class Main.java Menu 3 (Tarik Tunai) dan 6 (Cetak Mutasi)

a) Menu 3(Tarik Tunai)

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");
    } else {

        //minta pin
        System.out.print("Masukan PIN: ");
        String pinTarik = input.nextLine();
        //Cek pin
        if (akunAktif.otentikasi(pinTarik)) {
            System.out.print("Masukan nominal tarik: ");
        double tarik = input.nextDouble();
        input.nextLine();
        akunAktif.tarikTunai(tarik);
        }else {
            System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
    }
    }
    break;
```

Penjelasan

```
  if (akunAktif == null) {
      System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");
  } else {
```

Kode ini digunakan untuk mengecek apakah pengguna sudah memiliki akun aktif. Jika akunAktif null maka output akan keluar Error: Mohon Maaf, Anda belum memiliki nomor rekening!, namun jika akunAktif tidak null, maka program akan masuk ke bagian else dan melanjutkan proses penarikan.

```
//minta pin
System.out.print("Masukan PIN: ");
String pinTarik = input.nextLine();
```

Program meminta pengguna memasukkan PIN terlebih dahulu. PIN yang dimasukkan disimpan dalam variabel pinTarik, kemudian diperiksa menggunakan method:

- **akunAktif.otentikasi(pinTarik)**

Method otentikasi() akan membandingkan PIN yang dimasukkan dengan PIN yang tersimpan pada rekening. Jika PIN benar proses dilanjutkan namun jika salah akses ditolak.

```
        //Cek pin
        if (akunAktif.otentikasi(pinTarik)) {
            System.out.print("Masukan nominal tarik: ");
        double tarik = input.nextDouble();
        input.nextLine();
        akunAktif.tarikTunai(tarik);
        }else {
            System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
    }
    }
    break;
```

Kode ini digunakan untuk melanjutkan proses penarikan jika PIN benar dan menolak transaksi jika PIN salah.

- Jika **otentikasi(pinTarik)** menghasilkan true, program meminta pengguna memasukkan nominal penarikan.
- Nominal disimpan dalam variabel tarik dengan tipe data double.
- **akunAktif.tarikTunai(tarik)** digunakan untuk menjalankan proses penarikan pada rekening aktif.
- Jika PIN tidak sesuai, program masuk ke else dan menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

OUTPUT

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
3
Masukan PIN: 123456
Masukan nominal tarik: 120000
Transaksi Berhasil
```

b) Menu 6 (Cek Mutasi)

```
case 6:
    if(akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
    }else {

        //Masukan PIN
        System.out.print("Masukan PIN: ");
        String pinMutasi = input.nextLine();
        //Cek PIN
        if (akunAktif.otentikasi(pinMutasi)) {
            akunAktif.cetakMutasi();
        }else {
            System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
    }
    }
    break;
```

Penjelasan

```
if(akunAktif == null) {
    System.out.println("Error : Anda belum membuka rekening!");
}else {
```

Kode ini digunakan untuk mengecek apakah pengguna sudah memiliki rekening aktif. Jika akunAktif == null, berarti pengguna belum membuka rekening sehingga program menampilkan pesan error.

```
//Masukan PIN
System.out.print("Masukan PIN: ");
String pinMutasi = input.nextLine();
```

Program meminta pengguna memasukkan PIN dan menyimpannya ke variabel pinMutasi.

```
    //Cek PIN
    if (akunAktif.otentikasi(pinMutasi)) {
        akunAktif.cetakMutasi();
    }else {
        System.out.print("Akses Ditolak: PIN yang Anda masukkan salah!");
}
}
break;
```

Kode ini digunakan untuk menampilkan riwayat transaksi jika PIN benar dan menolak akses jika PIN salah.

- Jika PIN benar, akunAktif.cetakMutasi() dipanggil untuk menampilkan seluruh riwayat transaksi pada rekening.
- Jika PIN salah, program menampilkan pesan "Akses Ditolak: PIN yang Anda masukkan salah!".

OUTPUT

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
6
Masukan PIN: 123456
ID: TRX-T-1790521390192 | Jenis:Debit | Nominal: Rp120000.0
```

LAPORAN PRAKTIKUM PEKAN 4

PEMOGRAMAN BERORIENTASI OBJEK

Ekspansi Produk Keuangan (Inheritance)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah,S.ST., M.Kom.

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan sistem perbankan, terdapat berbagai jenis rekening yang memiliki karakteristik dan fitur yang berbeda. Contohnya adalah rekening tabungan yang memiliki fitur suku bunga, rekening giro yang memiliki fasilitas overdraft, serta rekening VIP yang dapat memiliki fitur tambahan tertentu. Jika setiap jenis rekening dibuat sebagai class yang berdiri sendiri, akan terdapat banyak kode yang sama dan dapat menyebabkan pengulangan kode.

Untuk mengatasi hal tersebut, digunakan konsep Inheritance (pewarisan) dalam Pemrograman Berorientasi Objek. Inheritance memungkinkan sebuah class baru mewarisi atribut dan method dari class yang sudah ada. Pada praktikum ini, class "Rekening" digunakan sebagai superclass, sedangkan "RekeningTabungan", "RekeningGiro", dan "RekeningVIP" dapat digunakan sebagai subclass yang memiliki fitur tambahan sesuai jenis rekeningnya.

Penerapan inheritance dilakukan menggunakan keyword "extends", sedangkan "super()" digunakan untuk memanggil constructor dari class induk. Selain itu, praktikum ini juga menerapkan konsep upcasting dan downcasting untuk mengelola berbagai jenis objek rekening dalam satu koleksi.

1.2 Tujuan

1. Memahami konsep Inheritance (pewarisan) dalam Pemrograman Berorientasi Objek.
2. Memahami perbedaan antara superclass dan subclass.
3. Menerapkan keyword "extends" untuk membuat class turunan dari class "Rekening".
4. Menerapkan "super()" untuk memanggil constructor dari class induk.

5. Membuat berbagai jenis rekening dengan fitur tambahan melalui inheritance.
6. Memahami dan menerapkan konsep upcasting dan downcasting pada objek rekening.
7. Mengembangkan program perbankan agar dapat menangani beberapa jenis produk rekening.

1.3 Manfaat

1. Mempermudah pembuatan class baru karena dapat menggunakan kembali atribut dan method dari superclass.
2. Mengurangi pengulangan kode sehingga program menjadi lebih terstruktur dan mudah dikembangkan.
3. Membantu memahami penerapan konsep inheritance dalam kasus yang lebih nyata, yaitu sistem perbankan.
4. Memahami cara membuat subclass dengan karakteristik dan fitur tambahan yang berbeda.
5. Memahami penggunaan upcasting dan downcasting dalam pengelolaan objek.
6. Meningkatkan kemampuan dalam merancang program Java yang menerapkan prinsip code reusability.

BAB II

PEMBAHASAN

2.1 Teori

Seiring berkembangnya sistem perbankan, bank tidak hanya memiliki satu jenis rekening. Ada "Rekening Tabungan" yang mana akan mendapat bunga bulanan dan "Rekening Giro" yang memiliki fasilitas pinjaman atau overdraft. Jika kita membuat class baru dari nol untuk masing masing entitas tersebut, kita akan melanggar prinsip DRY (Don't Repeat Yourself) karena menduplikasi banyak kode logika PIN dan mutasi. "Is-A" contohnya Tabungan is-a Rekening. langsung ke kelas induk:

• Konsep Pewarisan (Inheritance): Menurut Deitel dan Deitel (2017), pewarisan adalah bentuk software reuse di mana class baru diciptakan dengan menggunakan atribut dan behavior dari class yang sudah ada, lalu menambahkan kapabilitas baru. Hubungan ini dikenal sebagai relasi

• Penggunaan Keyword extends dan super : Di Java, subclass mewarisi kelas induk menggunakan kata kunci extends. Sementara itu, kata kunci super berfungsi sebagai jembatan

- super() digunakan di dalam konstruktor kelas anak untuk mengeksekusi inisialisasi konstruktor induk. Pemanggilan ini wajib berada di baris paling pertama di dalam konstruktor anak (Oracle, 2023).
- super.namaMethod() digunakan jika kelas anak perlu memanggil metode asli milik induk yang mungkin telah dimodifikasi (di-override).

• Pentingnya protected: Agar kelas anak (Tabungan, Giro) bisa memanipulasi saldo dan mencatat riwayatTransaksi tanpa harus membuka akses tersebut ke publik, atribut tersebut harus menggunakan modifier protected.

• Upcasting dan Downcasting: Karena adanya relasi Is-A, Java mengizinkan konversi tipe referensi antar hierarki kelas (Eckel, 2006):

• Upcasting (Implisit): Menyimpan objek kelas anak ke dalam variabel referensi kelas induk contohnya Rekening akun = new RekeningTabungan(...). Proses ini otomatis dan aman. Fungsinya sangat krusial agar kita bisa menyimpan berbagai jenis produk rekening ke dalam satu koleksi yang seragam seperti ArrayList tipe Rekening.

• Downcasting (Eksplisit): Memaksa referensi kelas induk kembali menjadi kelas anak spesifik contoh RekeningTabungan tab = (RekeningTabungan) akun;. Proses ini rawan memicu error ClassCastException saat program berjalan jika objek aslinya ternyata bukan Tabungan. Oleh karena itu, downcasting wajib didahului oleh pengecekan tipe menggunakan operator instanceof.

2.2 Langkah Langkah

1. Membuat Class Transaksi

1) Pertama saya membuat Package “prak4_Inheritance”
2) Selanjutnya dilanjukan dengan membuat Class Transaksi, Berikut Screenshot lengkap nya

```java
package prak4_Inheritance;

public class Transaksi {
    //mengubah atribut jd private
        private String idTransaksi;
        private String jenis;
        private double nominal;

        public Transaksi(String id, String jenis, double nominal) {
            this.idTransaksi = id;
            this.jenis = jenis;
            this.nominal = nominal;
        }
        // Hanya nyediakan getter (Read Only)
        public String getIdTransaksi() { return idTransaksi;}
        public String getJenis() { return jenis; }
        public double getNominal() { return nominal;}

        public void cetakDetail() {
            System.out.println("ID: " + idTransaksi + " | Jenis:"
            + jenis + " | Nominal: Rp" + nominal);
        }

    }
```

Penjelasan:

1) Buat class Transaksi

```
public class Transaksi {
```

Ini mengartikan kita sedang membuat sebuah class bernama Transaksi. Class bisa dianggap sebagai cetakan/model untuk membuat objek Transaksi.

2) Membuat State/Atribut

```
//mengubah atribut id private
private String idTransaksi;
private String jenis;
private double nominal;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class transaksi ini tersimpan data berupa

- idTransaksi dengan tipe data string
- jenis dengan tipe data string
- nominal dengan tipe data double.

Kata private digunakan agar atribut hanya dapat diakses secara langsung dari dalam class Transaksi. Hal ini merupakan penerapan konsep **enkapsulasi**.

3) Buat Constructor

```
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek Transaksi Ketika objek dibuat.

- idTransaksi di inisialisasikan dengan nilai id
- jenis diinisialisasikan dengan nilai jenis transaksi, yaitu jenis transaksi debit atau kredit
- nominal diinisialisasikan dengan nilai nominak transaksi

this digunakan untuk menunjukkan atribut yang berada di dalam class Transaksi.

4) Membuat Method Getter

```
// Hanya nyediakan getter (Read Only)
public String getIdTransaksi() { return idTransaksi;}
public String getJenis() { return jenis; }
public double getNominal() { return nominal;}
```

Method getter digunakan untuk mengambil atau membaca nilai atribut yang bersifat private.Pada class Transaksi terdapat tiga getter, yaitu:

- getIdTransaksi() digunakan untuk mengambil nilai idTransaksi.
- getJenis() digunakan untuk mengambil nilai jenis.
- getNominal() digunakan untuk mengambil nilai nominal.

Getter digunakan karena atribut dibuat private, sehingga nilai atribut tidak dapat diakses secara langsung dari luar class

5) Buat Method cetakDetail

```
    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis:"
        + jenis + " | Nominal: Rp" + nominal);
    }

}
```

Method cetakDetail() digunakan untuk menampilkan detail transaksi yang berisi ID transaksi, jenis transaksi, dan nominal transaksi. Program akan menampilkan informasi dengan format “ID: (ID transaksi) | Jenis: (jenis transaksi) | Nominal: Rp (nominal)”.

2. Membuat Class Rekening

Salin Code Program yang sudah dibuat pada praktikum prak2_Relasi, Dan ubah atau tambah beberapa bagian saja. Berikut Screenshot

```java
package prak4_Inheritance;
import java.util.ArrayList;

public class Rekening {
    //Batas atas dari class Rekening
    private String nomorRekening;
    private String namaPemilik;
    private String pin;

    //Gunakan protect agar subclass bisa mengakses langsung
    protected double saldo;
    protected ArrayList<Transaksi> riwayatTransaksi;

    //Isi konstruktor dan Method lain sama dengan modul 3
```

Penjelasan:

```java
ublic class Rekening {
  //Batas atas dari class Rekening
  private String nomorRekening;
  private String namaPemilik;
  private String pin;
```

Tiga atribut tersebut menggunakan private, sehingga hanya dapat diakses langsung dari dalam class Rekening.

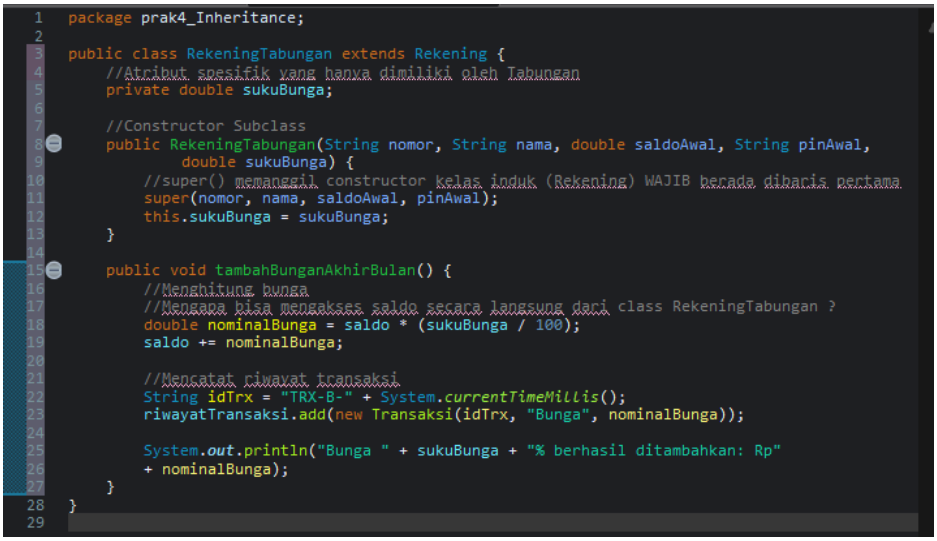
- nomorRekening → menyimpan nomor rekening.
- namaPemilik → menyimpan nama pemilik rekening.
- pin → menyimpan PIN rekening.

```java
//Gunakan protect agar subclass bisa mengakses langsung
protected double saldo;
protected ArrayList<Transaksi> riwayatTransaksi;
```

Kedua atribut menggunakan protected, sehingga dapat diakses oleh class Rekening dan subclass-nya.

- saldo → menyimpan jumlah saldo rekening dalam tipe double.
- riwayatTransaksi → menyimpan daftar riwayat transaksi menggunakan ArrayList dengan objek Transaksi.

3. Membuat Class RekeningTabungan



```
package prak4_Inheritance;

public class RekeningTabungan extends Rekening {
    //Atribut spesifik yang hanya dimiliki oleh Tabungan
    private double sukuBunga;

    //Constructor Subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal,
            double sukuBunga) {
        //super() memanggil constructor kelas induk (Rekening) WAJIB berada dibaris pertama
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBunganAkhirBulan() {
        //Menghitung bunga
        //Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        //Mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp"
        + nominalBunga);
    }
}
```

Penjelasan

```
public class RekeningTabungan extends Rekening {
```

Membuat RekeningTabungan sebagai subclass dari Rekening, sehingga dapat mewarisi atribut dan method dari class induk.

```
//Atribut spesifik yang hanya dimiliki oleh Tabungan
private double sukuBunga;
```

Menyimpan nilai suku bunga khusus untuk rekening tabungan.

```
//Constructor Subclass
public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal,
        double sukuBunga) {
    //super() memanggil constructor kelas induk (Rekening) WAJIB berada dibaris pertama
    super(nomor, nama, saldoAwal, pinAwal);
    this.sukuBunga = sukuBunga;
}
```

Merupakan constructor untuk mengisi data rekening dan suku bunga. super() digunakan untuk memanggil constructor class Rekening dan harus berada di baris pertama, sedangkan this.sukuBunga mengisi atribut suku bunga.

```
public void tambahBungaAkhirBulan() {
    //Menghitung bunga
    //Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
    double nominalBunga = saldo * (sukuBunga / 100);
    saldo += nominalBunga;
```

Menghitung nominal bunga berdasarkan saldo dan suku bunga, kemudian menambahkan bunga tersebut ke saldo. saldo dapat diakses langsung karena menggunakan protected pada class Rekening.

```
    //Mencatat riwayat transaksi
    String idTrx = "TRX-B-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
```

Membuat ID transaksi berdasarkan waktu dan mencatat transaksi bunga ke dalam riwayatTransaksi.

```
    System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp"
    + nominalBunga);


}
}
```

Menampilkan informasi bahwa bunga berhasil ditambahkan beserta nominalnya.

4. Membuat Class RekeningGiro

```
1  package prak4_Inheritance;
2
3  public class RekeningGiro extends Rekening {
4      private double batasOverdraft;
5
6      public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
7              double batasOverdraft) {
8          //memanggil inisialisasi dasar superclass
9          super(nomor, nama, saldoAwal, pinAwal);
10         this.batasOverdraft = batasOverdraft;
11     }
12
13     //Getter khusus Giro
14     public double getBatasOverdraft() {
15         return batasOverdraft;
16     }
17  // (Catatan: Penarikan hingga limit overdraft akan diselesaikan dimodul 5)
18 }
```

Penjelasan

```
public class RekeningGiro extends Rekening {
```

Membuat RekeningGiro sebagai subclass dari Rekening, sehingga mewarisi atribut dan method dari class induk.

```
private double batasOverdraft;
```

Menyimpan batas overdraft, yaitu batas saldo negatif yang dapat digunakan pada rekening giro.

```
public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
        double batasOverdraft) {
    //memanggil inisialisasi dasar superclass
    super(nomor, nama, saldoAwal, pinAwal);
    this.batasOverdraft = batasOverdraft;
}
```

Constructor digunakan untuk mengisi data dasar rekening melalui super() dan mengisi batasOverdraft dengan nilai yang diberikan.

```java
public double getBatasOverdraft() {
    return batasOverdraft;
}
```

Merupakan getter yang digunakan untuk mengambil nilai batas overdraft karena atribut batasOverdraft bersifat private.

TUGAS:

Perbarui Main.java agar mendukung :

1. Pembuatan Multi-Produk

Modifikasi case 1: Buka Rekening Baru di Main.java.

1) Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
2) Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
3) Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
4) Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).



Penjelasan:

```
case 1:
    System.out.print("Masukan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
```

Meminta input nomor rekening, nama pemilik, dan saldo awal. nextLine() digunakan untuk membaca teks, sedangkan nextDouble() untuk membaca angka**.**

```
// minta input PIN
System.out.print("Masukan PIN(6 digit):");
String pin = input.nextLine();
```

Meminta pengguna memasukkan PIN rekening.

```
//Opsi Produk
System.out.println("\nPiih Produk:");
System.out.println("1.Tabungan Umum");
System.out.println("2.Giro Bisnis");
System.out.print("Piih Produk: ");
int produk = input.nextInt();
input.nextLine();
Rekening rekeningBaru = null;
```

Menampilkan pilihan jenis rekening, yaitu Tabungan Umum atau Giro Bisnis, kemudian menyimpan pilihan pengguna ke variabel produk. Membuat variabel rekeningBaru bertipe Rekening yang awalnya bernilai null. Variabel ini nantinya dapat menyimpan objek RekeningTabungan atau RekeningGiro.

```
if (produk == 1) {
    System.out.print("Masukan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    System.out.println("Rekening Tabungan Berhasil dibuat.");
```

Jika pengguna memilih Tabungan, program meminta suku bunga lalu membuat objek RekeningTabungan dan menyimpannya ke rekeningBaru.

```
        System.out.println("Rekening Tabungan Berhasil dibuat.");
    } else if (produk == 2) {
        System.out.println("Masukan Batas Overdraft: ");
        double batasOverdraft = input.nextDouble();
        input.nextLine();

        rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
        System.out.println("Rekening Giro Berhasil dibuat.");
```

Jika memilih Giro, program meminta batas overdraft lalu membuat objek RekeningGiro dan menyimpannya ke rekeningBaru.

```
        System.out.println("Rekening Giro Berhasil dibuat.");
    } else {
        System.out.println("Pilihan produk tidak valid.");

    break;
    }

    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
```

Jika pilihan selain 1 atau 2, program menampilkan pesan bahwa pilihan tidak valid dan menghentikan proses case.

OUTPUT

```
Main (5) [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: nay
Masukan Saldo Awal: 200000
Masukan PIN(6 digit):123456

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 1
Masukan Suku Bunga (%): 2
Rekening atas nama nay berhasil dibuat.
Rekening Tabungan Berhasil dibuat.
```

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: nay
Masukan Saldo Awal: 200000
Masukan PIN(6 digit):123456

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
100000
Rekening atas nama nay berhasil dibuat.
Rekening Giro Berhasil dibuat.
```

2. Simulasi Waktu Akhir Bulan

1) Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2) Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
3) Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
4) Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```
        case 7:
            if(akunAktif == null) {
                System.out.println("Error : Anda belum membuka rekening!");
            } else if (akunAktif instanceof RekeningTabungan) {

                // DOWNCASTING
                RekeningTabungan tabungan =
                        (RekeningTabungan) akunAktif;

                tabungan.tambahBungaAkhirBulan();

            } else {

                System.out.println(
                        "Gagal: Fitur bunga akhir bulan "
                        + "hanya berlaku untuk Rekening Tabungan."
                );
            }

            break;
```

Penjelasan

```
case 7:
    if(akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
```

Mengecek apakah pengguna sudah memiliki rekening aktif. Jika belum, program menampilkan pesan error.

```
} else if (akunAktif instanceof RekeningTabungan) {

    // DOWNCASTING
    RekeningTabungan tabungan =
            (RekeningTabungan) akunAktif;

    tabungan.tambahBungaAkhirBulan();
```

Mengecek apakah rekening aktif merupakan RekeningTabungan menggunakan instanceof. Jika iya, dilakukan downcasting dari tipe Rekening menjadi RekeningTabungan, kemudian method tambahBungaAkhirBulan() dijalankan untuk menambahkan bunga.

```
} else {

    System.out.println(
            "Gagal: Fitur bunga akhir bulan "
            + "hanya berlaku untuk Rekening Tabungan."
    );
}

break;
```

Jika rekening aktif bukan RekeningTabungan, fitur bunga tidak dapat digunakan karena hanya berlaku untuk rekening tabungan.

OUTPUT

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
0.Keluar
Pilih Menu:
7
Bunga 2.0% berhasil ditambahkan: Rp4000.0
```

LAPORAN PRAKTIKUM PEKAN 5

PEMOGRAMAN BERORIENTASI OBJEK

Transaksi Dinamis (Polymorphism)



Oleh:
Naysha Aprilia Rizki
2511531014
Kelas B

Dosen Pengampu:
Nurfiah,S.ST., M.Kom.

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan sistem perbankan, setiap jenis rekening dapat memiliki aturan transaksi yang berbeda. Contohnya, rekening tabungan memiliki aturan penarikan berdasarkan saldo yang tersedia, sedangkan rekening giro memungkinkan penarikan melebihi saldo hingga batas overdraft tertentu. Jika setiap jenis rekening memiliki implementasi transaksi yang berbeda, diperlukan cara agar program dapat menjalankan aturan tersebut secara fleksibel.

Salah satu konsep Pemrograman Berorientasi Objek yang dapat digunakan adalah *Polymorphism* atau polimorfisme. Konsep ini memungkinkan method dengan nama yang sama memiliki perilaku berbeda sesuai dengan jenis objeknya. Dalam praktikum ini, polimorfisme diterapkan melalui *Method Overriding* pada method tarikTunai() di kelas RekeningGiro, serta *Dynamic Method Dispatch* untuk menentukan method yang dijalankan saat program berlangsung. Selain itu, konsep ini digunakan dalam pengembangan fitur transfer antar rekening.

1.2 Tujuan

1. Memahami konsep polimorfisme dalam Pemrograman Berorientasi Objek.
2. Memahami penerapan *Method Overriding* pada kelas turunan.
3. Memahami konsep *Dynamic Method Dispatch* dalam Java.
4. Menerapkan anotasi `@Override` pada method yang ditimpa.
5. Mengembangkan fitur transaksi yang dapat menyesuaikan jenis rekening.
6. Menerapkan polimorfisme dalam pembuatan fitur transfer antar rekening.

1.3 Manfaat

1. Memahami penerapan polimorfisme dalam pemrograman Java.
2. Mengurangi kebutuhan penggunaan percabangan untuk menangani perilaku objek yang berbeda.
3. Mempermudah pengembangan program dengan aturan transaksi yang beragam.
4. Meningkatkan pemahaman tentang penggunaan *Method Overriding* dan *Dynamic Method Dispatch*.
5. Melatih kemampuan dalam mengembangkan fitur transaksi perbankan menggunakan konsep Pemrograman Berorientasi Objek.

BAB II

PEMBAHASAN

2.1 Teori

Polimorfisme merupakan salah satu konsep dalam Pemrograman Berorientasi Objek yang memungkinkan satu method memiliki perilaku berbeda sesuai dengan jenis objek yang menjalankannya. Dalam sistem perbankan, konsep ini dapat diterapkan pada proses penarikan tunai, ketika rekening biasa dan rekening giro memiliki aturan penarikan yang berbeda.

1. **Polimorfisme (*Polymorphism*)**

Polimorfisme berarti memiliki banyak bentuk. Dalam Java, konsep ini memungkinkan pemanggilan method yang sama menghasilkan perilaku berbeda berdasarkan jenis objek sebenarnya saat program dijalankan. Pada sistem perbankan, method tarikTunai() dapat memiliki aturan berbeda untuk rekening biasa dan rekening giro.

2. **Method Overriding**

Method Overriding adalah proses ketika kelas turunan mendefinisikan kembali method yang berasal dari kelas induk dengan nama dan parameter yang sesuai. Dalam praktikum ini, kelas RekeningGiro menimpa method tarikTunai() agar penarikan dapat dilakukan hingga batas overdraft yang ditentukan.

3. **Dynamic Method Dispatch**

Dynamic Method Dispatch merupakan mekanisme Java dalam menentukan method yang akan dijalankan berdasarkan jenis objek sebenarnya saat program berjalan. Dengan mekanisme ini, pemanggilan akunAktif.tarikTunai() dapat menjalankan

implementasi milik Rekening atau RekeningGiro sesuai objek yang digunakan, tanpa perlu membuat percabangan khusus untuk setiap jenis rekening.

4. **Anotasi @Override**

Anotasi @Override digunakan untuk menandai bahwa suatu method pada kelas turunan dimaksudkan untuk menimpa method dari kelas induk. Anotasi ini membantu compiler mendeteksi kesalahan, seperti ketidaksesuaian nama atau parameter method.

5. **Penerapan Polimorfisme pada Fitur Transfer**

Polimorfisme juga dapat diterapkan pada fitur transfer antar rekening melalui method transfer(Rekening tujuan, double nominal, String pinInput). Parameter Rekening tujuan memungkinkan method menerima objek dari berbagai kelas turunan rekening. Proses transfer dilakukan melalui autentikasi PIN, penarikan dana dari rekening pengirim, dan penyetoran dana ke rekening tujuan apabila penarikan berhasil.

2.2 Langkah Langkah

Praktikum 5 ini, melanjutkan yang sudah dibuat dari praktikum 1 sampai praktikum 4, hanya menambahkan beberapa bagian di beberapa class saja. Class yang digunakan tetap sama seperti Praktikum sebelumnya yaitu Class Transaksi, Class Rekening, Class Rekening Tabungan, Class Rekening Giro, Class Main. Dan yang akan diubah yaitu class yang akan saya jelaskan:

1. **Membuat Class Rekening**

   Salin Code Program yang sudah dibuat pada praktikum prak4_Inheritance, Dan ubah dibagian tarikTunai saja. Berikut Screenshot

```
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
        return false;
        } else if (nominal > saldo) {
            System.out.println("Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp" + saldo);
            return false;
        } else {
            saldo -= nominal;

    String idTrx = "TRX-T-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

    System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
    return true; //transaksi sukses
        }
}
```

Penjelasan:

```
public boolean tarikTunai(double nominal) {
```

Diubah dari public void tarikTunai(double nominal) menjadi public boolean tarikTunai(double nominal) karena pada Praktikum 5, method tarikTunai() harus mengembalikan status berhasil(true) atau gagalnya(false) transaksi untuk mendukung fitur transfer antar rekening.

```
if (nominal <= 0) {
    System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
    return false;
```

Jika nominal kecil sama 0 maka Transaksi gagal dan akan mengeluarkan output “Gagal: Nominal tarik harus lebih dari 0!”, kemudian mengembalikan nilai false sebagai tanda bahwa transaksi gagal.

```
} else if (nominal > saldo) {
    System.out.println("Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp" + saldo);
    return false;
```

Selain itu, Jika nominal yang diinputkan lebih dari saldo maka transaksi akan gagal, dan akan mengeluarkan output “Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp” + jumlah saldo yang berada pada rekening, kemudian mengembalikan nilai false sebagai tanda bahwa transaksi gagal.

```
        } else {
            saldo -= nominal;

    String idTrx = "TRX-T-" + System.currentTimeMillis();
    riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

    System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
    return true; //transaksi sukses
        }
}
```

Jika nominal penarikan tidak melebihi saldo rekening dan lebih dari 0, maka transaksi akan berhasil. Saldo rekening akan dikurangi sesuai

nominal yang ditarik. Selanjutnya, sistem akan membuat ID transaksi dengan awalan “TRX-T-” berdasarkan waktu saat ini dan menyimpan data transaksi ke dalam riwayat dengan jenis “Debit”. Setelah itu, program akan menampilkan pesan bahwa penarikan berhasil beserta sisa saldo rekening, kemudian mengembalikan nilai true sebagai tanda bahwa transaksi berhasil.

2. Class RekeningGiro

```
package Prak5_Polymorphism;

public class RekeningGiro extends Rekening {
        private double batasOverdraft;

        public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
                double batasOverdraft) {
            //memanggil inisialisasi dasar superclass
            super(nomor, nama, saldoAwal, pinAwal);
            this.batasOverdraft = batasOverdraft;
        }

        // IMPLEMENTASI POLIMORFISME (OVERRIDING)
        @Override
        public boolean tarikTunai(double nominal) {
            if (nominal <= 0) {
                System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
                return false;
            }
            // Logika khusus Giro: Boleh ditarik asalkan nominal tidak melebihi(saldo + batasOverdraft)
            else if (nominal > (saldo + batasOverdraft)) {
                System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
                return false;
            }
            else {
                saldo -= nominal; //Saldo bisa bernilai negatif di sini (memakan overdraft)

                String idTrx = "Trx-G-" + System.currentTimeMillis();
                riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

                System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
                return true;
            }
        }
        //Getter khusus Giro
        public double getBatasOverdraft() {
            return batasOverdraft;
        }
    }
```

Penjelasan

```
public class RekeningGiro extends Rekening {
```

Membuat RekeningGiro sebagai subclass dari Rekening, sehingga mewarisi atribut dan method dari class induk.

```
private double batasOverdraft;
```

Menyimpan batas overdraft, yaitu batas saldo negatif yang dapat digunakan pada rekening giro.

```
        // IMPLEMENTASI POLIMORFISME (OVERRIDING)
        @Override
        public boolean tarikTunai(double nominal) {
            if (nominal <= 0) {
                System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
                return false;
            }
            // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi(saldo + batasOverdraft)
            else if (nominal > (saldo + batasOverdraft)) {
                System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft));
                return false;
            }
            else {
                saldo -= nominal; //Saldo bisa bernilai negatif di sini (memakan overdraft)

                String idTrx = "Trx-G-" + System.currentTimeMillis();
                riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

                System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
                return true;
            }
        }
        //Getter khusus Giro
```

method tarikTunai() pada class RekeningGiro digunakan untuk melakukan penarikan uang dengan aturan khusus, yaitu pengguna dapat menarik uang melebihi saldo yang tersedia selama jumlah penarikan tidak melampaui batas overdraft. Method ini juga memvalidasi nominal penarikan, mengurangi saldo, mencatat transaksi ke dalam riwayat, menampilkan hasil transaksi, serta mengembalikan nilai true jika berhasil dan false jika gagal. Penerapan @Override menunjukkan konsep polimorfisme, sehingga rekening giro memiliki aturan penarikan yang berbeda dari rekening biasa. Penjelasannya:

```
    // IMPLEMENTASI POLIMORFISME (OVERRIDING)
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
```

@Override digunakan untuk menandakan bahwa method tarikTunai() pada class RekeningGiro menimpa method milik class induk Rekening. Method ini menggunakan tipe boolean agar dapat mengembalikan nilai true jika transaksi berhasil dan false jika gagal.

```
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik Giro harus lebih dari 0!");
        return false;
    }
    // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi(saldo + batasOver
```

Jika nominal yang diinputkan kurang dari atau sama dengan 0, maka transaksi akan gagal dan program menampilkan pesan bahwa nominal penarikan harus lebih dari 0. Nilai false dikembalikan untuk menandakan bahwa transaksi tidak berhasil.

```
    // Logika Khusus Giro: Boleh ditarik asalkan nominal tidak melebihi(saldo + batasOverdraft)
    else if (nominal > (saldo + batasOverdraft)) {
        System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp" + (saldo + batasOverdraft)
        return false;
    }
    else {
```

Jika nominal penarikan melebihi jumlah saldo ditambah batas overdraft, maka transaksi akan gagal. Program menampilkan pesan bahwa penarikan melebihi batas overdraft beserta jumlah maksimal penarikan yang diperbolehkan, kemudian mengembalikan nilai false.

```
    else {
        saldo -= nominal; //Saldo bisa bernilai negatif di sini (memakan overdraft)

        String idTrx = "Trx-G-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit Giro", nominal));

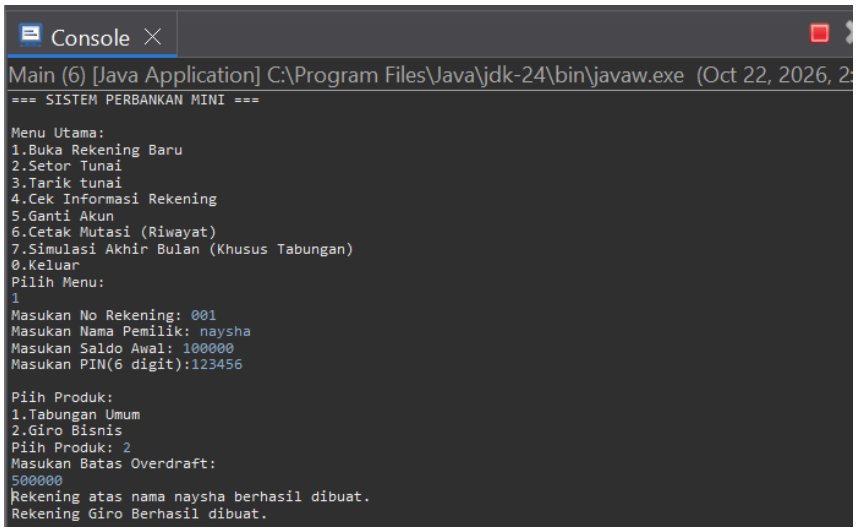
        System.out.println("Tarik tunai Giro Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
        return true;
    }
}
```

Jika nominal penarikan memenuhi ketentuan batas overdraft, maka saldo rekening akan dikurangi sesuai nominal yang ditarik. Selanjutnya, sistem akan membuat ID transaksi dengan awalan "Trx-G-" berdasarkan waktu saat ini dan menyimpan data transaksi ke dalam riwayat dengan jenis "Debit Giro". Setelah itu, program menampilkan pesan bahwa penarikan berhasil beserta sisa saldo rekening, kemudian mengembalikan nilai true sebagai tanda bahwa transaksi berhasil.

```
        //Getter khusus Giro
        public double getBatasOverdraft() {
            return batasOverdraft;
        }
    }
```

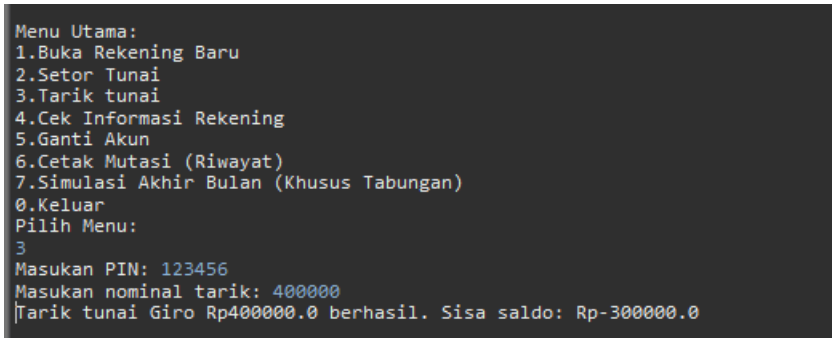
Merupakan getter yang digunakan untuk mengambil nilai batas overdraft karena atribut batasOverdraft bersifat private.

OUTPUT:



Pilih 1, untuk membuka rekening baru dan masukan saldo awal sebesar Rp 100.000.

Pilih Produk 2, yaitu Giro Bisnis masukan batas Overdraftnya Rp 500.000



Pilih menu 3, yaitu Tarik Tunai. Masukan nominal nya Rp 400.000 Karna transaksi ini di Giro tetap bisa dilakukan tapi Sisa saldo jadi Rp -300.000.

LATIHAN

1. Buat method Transfer di class Rekening, tambahkan satu method bar dengan nama : public void transfer(Rekening tujuan, double nominal, String pinInput)

• Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.

• Logika Method

▪ Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.

▪ Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).

▪ Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.

• Minta pengguna memasukkan Nomor Rekening Tujuan.

• Cari nomor tersebut di dalam daftarRekening.

• Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Jawab:

1. Method transfer di class Rekening

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("PIN salah!");
        return;
    }

    if (this.tarikTunai(nominal)) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer berhasil!");
    }
}
```

Penjelasan:

```
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    if (!otentikasi(pinInput)) {
        System.out.println("PIN salah!");
        return;
    }
```

Jika PIN yang dimasukkan tidak sesuai, maka program akan menampilkan pesan “PIN salah!” dan menghentikan proses transfer menggunakan return, sehingga penarikan dan penyetoran ke rekening tujuan tidak dilakukan.

```
    if (this.tarikTunai(nominal)) {
        tujuan.setorTunai(nominal);
        System.out.println("Transfer berhasil!");
    }
}
```

Jika penarikan uang berhasil dan method tarikTunai(nominal) mengembalikan nilai true, maka program akan menyetorkan uang ke rekening tujuan melalui method setorTunai(nominal) dan menampilkan pesan “Transfer berhasil!”. Jika penarikan gagal, maka penyetoran tidak dilakukan dan pesan keberhasilan tidak ditampilkan.

2. Menu Main

1) Tambahkan Menu 8. Transfer ke Rekening Lain

```
System.out.println("8. Transfer ke Rekening Lain");
```

2) Case 8

```
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan Nomor Rekening Tujuan: ");
        String nomorTujuan = input.nextLine();

        Rekening akunTujuan = null;

        for (Rekening rekening : daftarRekening) {
            if (rekening.getNomorRekening().equals(nomorTujuan)) {
                akunTujuan = rekening;
                break;
            }
        }

        if (akunTujuan == null) {
            System.out.println("Rekening tujuan tidak ditemukan!");
        } else if (akunTujuan == akunAktif) {
            System.out.println("Tidak dapat transfer ke rekening sendiri!");
        } else {
            System.out.print("Masukkan nominal transfer: ");
            double nominal = input.nextDouble();
            input.nextLine();

            System.out.print("Masukkan PIN: ");
            String pinTransfer = input.nextLine();

            akunAktif.transfer(akunTujuan, nominal, pinTransfer);
        }
    }
    break;
```

Penjelasan:

```
if (akunAktif == null) {
    System.out.println("Error: Anda belum membuka rekening!");
```

Jika pengguna memilih menu 8, program akan memeriksa apakah rekening aktif tersedia. Jika belum, program menampilkan pesan “Error: Anda belum membuka rekening

```
} else {
    System.out.print("Masukkan Nomor Rekening Tujuan: ");
    String nomorTujuan = input.nextLine();

    Rekening akunTujuan = null;

    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(nomorTujuan)) {
            akunTujuan = rekening;
            break;
        }
    }
```

Jika sudah, pengguna diminta memasukkan nomor rekening tujuan, kemudian program mencari nomor tersebut di dalam daftarRekening

```
  if (akunTujuan == null) {
      System.out.println("Rekening tujuan tidak ditemukan!");
```

Jika rekening tujuan tidak ditemukan, program menampilkan pesan "Rekening tujuan tidak ditemukan!"

```
} else if (akunTujuan == akunAktif) {
    System.out.println("Tidak dapat transfer ke rekening sendiri!");
```

Jika rekening tujuan sama dengan rekening aktif, program menampilkan pesan "Tidak dapat transfer ke rekening sendiri!".

```
    } else {
        System.out.print("Masukkan nominal transfer: ");
        double nominal = input.nextDouble();
        input.nextLine();

        System.out.print("Masukkan PIN: ");
        String pinTransfer = input.nextLine();

        akunAktif.transfer(akunTujuan, nominal, pinTransfer);
    }
}
break;
```

Jika rekening tujuan valid, program meminta nominal transfer dan PIN, kemudian memanggil method transfer() untuk memproses transaksi.

OUTPUT:

1. Buat 2 Rekening, 001 dan 002

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: naysha
Masukan Saldo Awal: 200000
Masukan PIN(6 digit):123456

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
500000
Rekening atas nama naysha berhasil dibuat.
Rekening Giro Berhasil dibuat.
```

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 002
Masukan Nama Pemilik: acha
Masukan Saldo Awal: 300000
Masukan PIN(6 digit):130407

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
500000
Rekening atas nama acha berhasil dibuat.
Rekening Giro Berhasil dibuat.
```

2. Lakukan Transfer ke Rekening Lain

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
8
Masukkan Nomor Rekening Tujuan: 001
Masukkan nominal transfer: 100000
Masukkan PIN: 130407
Tarik tunai Giro Rp100000.0 berhasil. Sisa saldo: Rp200000.0
Setor tunai Rp100000.0 berhasil Saldo saat ini: Rp300000.0
Transfer berhasil!

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
```

LINK TUGAS VIDEO :

https://youtu.be/F2074W0q4Sk?si=EfkfxtiJgjZPRhpz